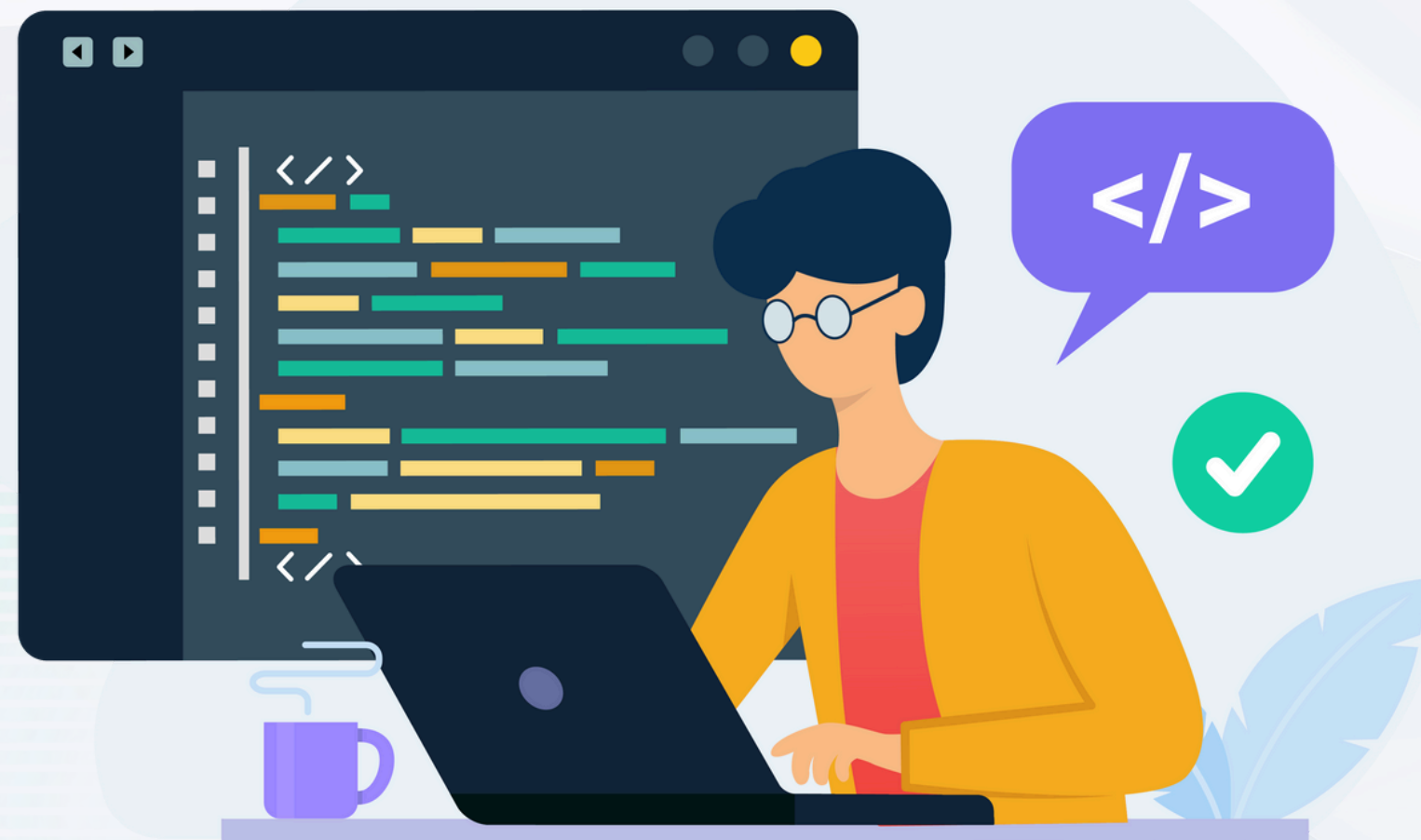




MODUL 2 DATA MINING

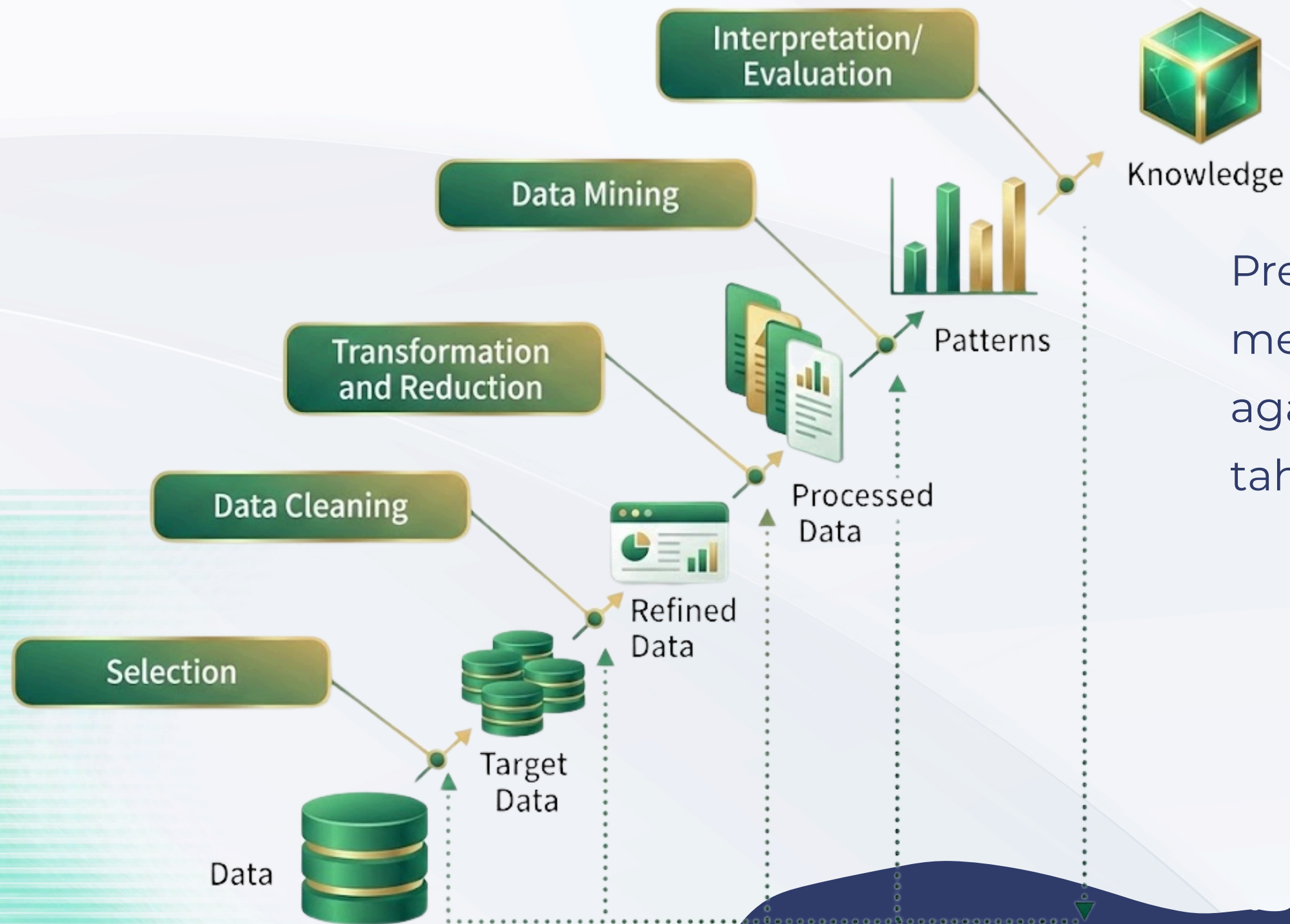
Data Preprocessing (Konsep
dan Praktikum Dasar)



TUJUAN

Pertemuan ini bertujuan membekali peserta dengan pemahaman konseptual serta pengalaman awal dalam melakukan data preprocessing menggunakan tools sederhana. Peserta diharapkan memahami bahwa data preprocessing merupakan fondasi utama sebelum menerapkan algoritma Data Mining, karena kualitas data sangat menentukan kualitas hasil analisis dan model.

Peran Data Preprocessing dalam Data Mining



Preprocessing berfungsi menyiapkan data mentah agar layak dianalisis pada tahap Data Mining.

CONTOH KEGAGALAN MODEL AKIBAT DATA YANG BURUK

- Data duplikat menyebabkan bias pada model
- Missing values menurunkan akurasi prediksi
- Outlier ekstrem mengganggu perhitungan jarak (misalnya pada K-Means)
- Perbedaan skala data menyebabkan atribut tertentu mendominasi model



KARAKTERISTIK DATA DUNIA NYATA

Data pada dunia nyata jarang berada dalam kondisi ideal. Beberapa karakteristik umum antara lain:

- Data tidak lengkap (missing values)
 - Contoh: kolom umur atau pendapatan kosong.
- Data tidak konsisten
 - Contoh: penulisan “Jakarta”, “DKI Jakarta”, dan “JKT”.
- Data ganda (duplicate records)
 - Satu entitas tercatat lebih dari satu kali.
- Data ekstrem (outlier)
 - Nilai sangat jauh dari mayoritas data.
- Perbedaan skala dan format data
 - Contoh: pendapatan (jutaan) dan usia (puluhan).

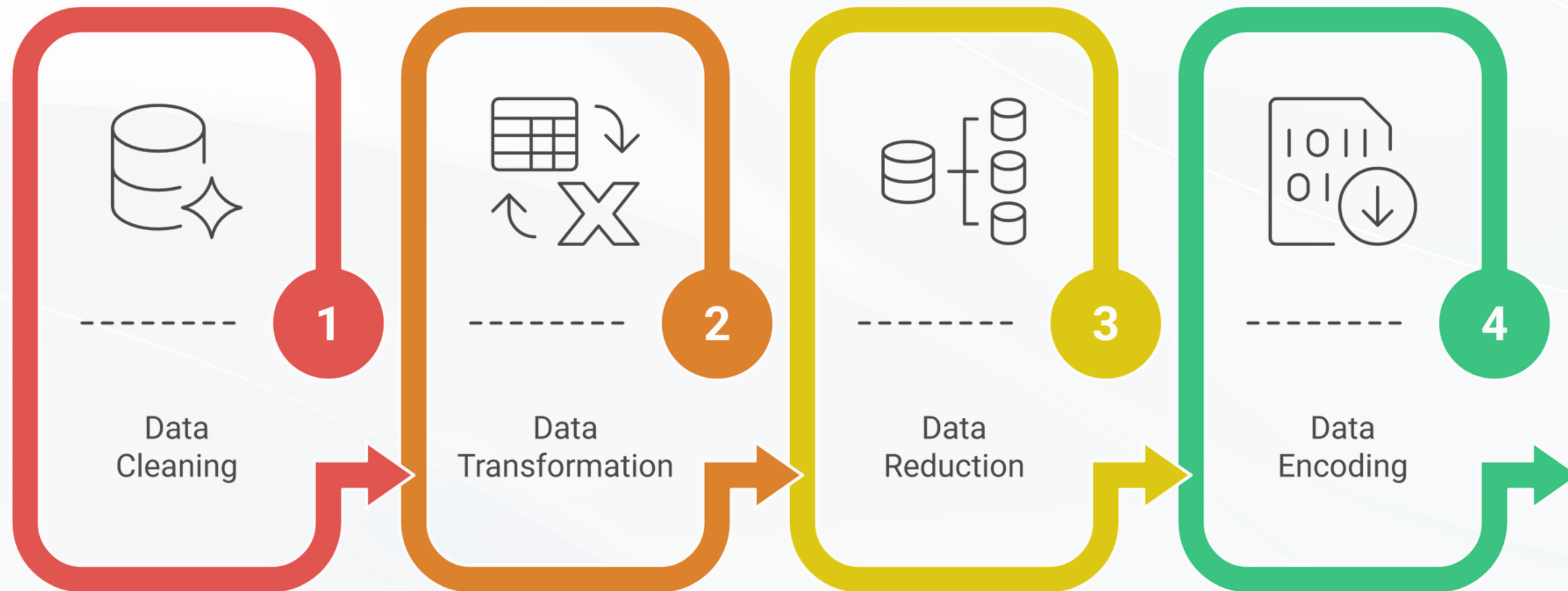


PERMASALAHAN UMUM PADA DATASET

Permasalahan yang sering ditemui dalam dataset:

- Nilai kosong dan tidak valid (NULL, 0 tidak logis, simbol tertentu)
- Kesalahan input data (human error)
- Variasi format data:
 - Tanggal: DD-MM-YYYY, YYYY/MM/DD
 - Numerik: pemisah ribuan dan desimal
 - Teks: huruf besar/kecil tidak konsisten
- Distribusi data tidak seimbang (imbalanced data)

TAHAPAN UTAMA DATA PREPROCESSING



Pendekatan KDD membantu meminimalkan risiko tersebut dengan menyediakan kerangka kerja sistematis dalam pengolahan data skala besar

Library	Kegunaan Utama
NumPy	Operasi numerik, array, dan matriks
Pandas	Manipulasi data tabel (DataFrame), load dataset
Matplotlib Matplotlib.pyplot	Visualisasi data dasar
Seaborn	Visualisasi statistik dan korelasi
Scikit-learn	Algoritma klasifikasi, regresi, clustering, preprocessing, evaluasi model
Statsmodels	Analisis statistik dan regresi
mlxtend	Association Rule Mining (Apriori, FP-Growth)

PRAKTIK SEDERHANA

Eksplorasi Data Awal (EDA)

Import Library

```
import pandas as pd  
import numpy as np  
import Matplotlib.pyplot as plt
```

Memuat Dataset (Load Data)

```
data = pd.read_csv("dataset_latihan.csv")  
data
```

Penjelasan:

- Dataset dimuat ke dalam objek *DataFrame*, yaitu struktur data utama dalam *Pandas*.

Melihat Dimensi Dataset

```
data.shape
```

Penjelasan:

- Output berupa (*jumlah_baris*, *jumlah_kolom*)
- Membantu memahami skala dataset

Melihat Nama Kolom

```
data.columns
```

Penjelasan:

- Menampilkan seluruh atribut/fitur yang terdapat dalam dataset.

PRAKTIK SEDERHANA

Eksplorasi Data Awal (EDA)

Memeriksa Struktur dan Tipe Data

```
data.info()
```

Penjelasan:

Digunakan untuk:

- Mengetahui tipe data setiap kolom
- Mendeteksi adanya nilai kosong (missing values)
- Mengetahui jumlah data non-null

Tahap ini sangat penting dalam Data Understanding.

Statistik Deskriptif Data Numerik

```
data.describe(include="all")
```

Penjelasan:

Menampilkan ringkasan statistik, seperti:

- Mean
- Standar deviasi
- Nilai minimum dan maksimum
- Kuartil

Digunakan untuk memahami distribusi data numerik.

PRAKTIK SEDERHANA

Eksplorasi Data Awal (EDA)

Mengidentifikasi Data Kategorikal

```
data.select_dtypes(include=['object']).columns
```

Penjelasan:

- Digunakan untuk mengetahui kolom bertipe kategorikal (teks), yang biasanya memerlukan proses encoding pada tahap selanjutnya

Menghitung rata-rata nilai numerik berdasarkan kategori

```
data.groupby('Kolom_Kategorial')['Kolom_Numerik'].mean()
```

Penjelasan:

- Memberikan gambaran awal hubungan antar variabel.

Menghitung jumlah data per kategori

```
data['Kolom_Kategorial'].value_counts()
```

Penjelasan:

- Digunakan untuk melihat distribusi data pada kolom kategorikal.

PRAKTIK SEDERHANA

PEMERIKSAAN & PENANGANAN MASALAH DATA

Mengecek Nilai Kosong (Missing Values)

```
df.isnull().sum()
```

Penjelasan kode:

- *isnull()* → mendeteksi nilai kosong (NaN)
- *sum()* → menghitung jumlah data hilang per kolom

Strategi Penanganan Missing Values

Dalam preprocessing, missing values dapat ditangani dengan:

1. Menghapus data (jika jumlahnya sedikit)
2. Mengisi data dengan:
 - Mean → untuk data numerik terdistribusi normal
 - Median → untuk data numerik dengan outlier
 - Mode → untuk data kategorikal

```
df = df.dropna()
```

```
df["kolom"] = df["kolom"].fillna(df["kolom"].mean())
```

```
df["kolom"] = df["kolom"].fillna(df["kolom"].median())
```

```
df["kolom"] = df["kolom"].fillna(df["kolom"].mode()[0])
```

```
df["kolom"] = pd.to_numeric(df["kolom"], errors="coerce")
```

merubah teks menjadi numerik

```
df["kolom"] = df["kolom"].apply(lambda x: max(x, 0))
```

merubah minus menjadi nol

PRAKTIK SEDERHANA

PEMERIKSAAN & PENANGANAN MASALAH DATA

Pengelompokan Umur

1. Buat fungsi pengelompokan

```
def kelompokkan_umur(x):
```

```
    if x <= 17:
```

```
        return "Anak-anak"
```

```
    elif x <= 25:
```

```
        return "Remaja"
```

```
    elif x <= 55:
```

```
        return "Dewasa"
```

```
    else:
```

```
        return "Lansia"
```

2. Terapkan fungsi tersebut ke kolom baru

menggunakan .apply()

```
df['kategori_umur'] =
```

```
df['umur'].apply(kelompokkan_umur)
```

```
df['kategori_umur'].value_counts()
```

Standarisasi Tanggal

```
df["tanggal_transaksi"]
```

```
pd.to_datetime(df["tanggal_transaksi"],  
errors="coerce")
```

PRAKTIK SEDERHANA

PEMERIKSAAN & PENANGANAN MASALAH DATA

Inkonsistensi Kategori

```
df["kolom"].unique()
```

Penjelasan:

- *unique() → menampilkan semua nilai unik*
- *Membantu mendeteksi teks atau nilai ekstrem*

Membersihkan Inkonsistensi

```
df["kolom"] = df["kolom"].str.title()
```

Penjelasan:

- *str.title() → menyeragamkan format teks*
- *Mengubah jakarta, JAKARTA menjadi Jakarta*

```
df["kolom"] = df["kolom"].replace("", "Tidak Diketahui")
```

kondisi jika ada kolom kosong

```
df["kolom"] = df["kolom"].apply(lambda x: max(x, 0))
```


PRAKTIK SEDERHANA

PEMERIKSAAN & PENANGANAN MASALAH DATA

Mengecek Duplikasi

```
df.duplicated().sum()
```

Penjelasan:

- *duplicated()* → menandai baris duplikat
- *sum()* → menghitung jumlah duplikasi

Menghapus Duplikasi

```
df = df.drop_duplicates()
```

Penjelasan:

- *Menghapus baris data yang identik*
- *Mengurangi bias pada analisis*

```
df['kolom'] = df['kolom'].apply(lambda x: max(x, 0))
```

PRAKTIK SEDERHANA

PEMERIKSAAN & PENANGANAN MASALAH DATA

Deteksi Outlier Total Transaksi (Boxplot)

```
plt.boxplot(df["total_transaksi"].dropna())  
plt.yscale("log")  
plt.title("Boxplot Total Transaksi (Log Scale)")  
plt.ylabel("Total Transaksi (log)")  
plt.show()
```

`df["total_transaksi"].describe()`

cek statistik

mengatasi Outlier Total Transaksi (Boxplot)

```
Q1 = df["total_transaksi"].quantile(0.25)  
Q3 = df["total_transaksi"].quantile(0.75)  
IQR = Q3 - Q1  
  
upper_bound = Q3 + 1.5 * IQR  
df.loc[df["total_transaksi"] > upper_bound,  
"total_transaksi"] = upper_bound
```

Penjelasan:

- *Q1 → kuartil bawah (25%)*
- *Q3 → kuartil atas (75%)*
- *IQR → rentang data tengah*
- *$Q3 + 1.5 \times IQR$ → batas atas nilai wajar*
- *Nilai di atas batas tersebut dibatasi (bukan dihapus)*

PRAKTIK SEDERHANA

ENCODING ATRIBUT KATEGORIKAL

Identifikasi Atribut Kategorikal

```
df.select_dtypes(include="object").columns
```

Penjelasan kode:

- `select_dtypes(include="object")` → memilih kolom bertipe teks
- `.columns` → menampilkan nama kolom

Import Library

```
from sklearn.preprocessing import LabelEncoder
```

Penjelasan:

- Mengimpor kelas `LabelEncoder` dari `scikit-learn`
- Digunakan untuk mengubah kategori → angka

Encoding

```
le_jk = LabelEncoder()  
df["jenis_kelamin_enc"] =  
le_jk.fit_transform(df["jenis_kelamin"])
```

Penjelasan baris per baris:

- `LabelEncoder()` → membuat objek encoder
- `fit_transform()`:
 - *fit* → mempelajari kategori unik
 - *transform* → mengonversi kategori menjadi angka
- Hasil disimpan di kolom baru `jenis_kelamin_enc`

PRAKTIK SEDERHANA

NORMALISASI DATA NUMERIK

Mengapa Normalisasi Diperlukan?

Normalisasi digunakan untuk menyeragamkan skala data numerik, karena pada dataset Anda terdapat perbedaan skala yang signifikan, misalnya:

- umur → puluhan
- total_transaksi → jutaan
- hasil encoding → bilangan kecil (0, 1, 2, ...)

Tanpa normalisasi:

- Algoritma berbasis jarak (K-Means, KNN) akan lebih dipengaruhi oleh fitur berskala besar
- Fitur berskala kecil menjadi tidak berpengaruh

Identifikasi Atribut Numerik yang Akan Dinormalisasi

```
df.select_dtypes(include=["int64", "float64"]).columns
```

Dalam praktikum ini, yang dinormalisasi adalah fitur numerik utama, bukan ID.

PRAKTIK SEDERHANA

NORMALISASI DATA NUMERIK

Menentukan Kolom yang Akan Dinormalisasi

```
kolom_numerik = [  
    "umur", "total_transaksi", "jenis_kelamin_enc", "jenis_produk_enc", "kota_enc", "nama_produk_enc"]
```

Metode Normalisasi yang Digunakan Untuk praktikum dasar, digunakan:

Min-Max Normalization

$$X_{norm} = \frac{X - X_{min}}{X_{max} - X_{min}}$$

Hasil:

- Nilai berada pada rentang 0 – 1
- Mudah dipahami dan divisualisasikan

PRAKTIK SEDERHANA

NORMALISASI DATA NUMERIK

Import Library

```
from sklearn.preprocessing import MinMaxScaler
```

Proses Normalisasi

```
scaler = MinMaxScaler()  
df_normalized = df.copy()  
df_normalized[kolom_numerik] = scaler.fit_transform(df[kolom_numerik])
```

Penjelasan baris per baris:

- *df.copy()* → menjaga dataset asli tetap utuh
- *fit_transform()*:
 - *fit* → menghitung nilai min dan max
 - *transform* → menerapkan rumus normalisasi
- Hasil ditimpa ke kolom numerik

PRAKTIK SEDERHANA

NORMALISASI DATA NUMERIK

Validasi Hasil Normalisasi

```
df_normalized[kolom_numerik].describe()
```

Yang diharapkan:

- Semua nilai berada di rentang 0 – 1

Bandingkan Sebelum dan Sesudah

```
df[["umur", "total_transaksi"]].head()  
df_normalized[["umur", "total_transaksi"]].head()
```

Catatan:

1. *Normalisasi tidak mengubah pola data*
2. *Normalisasi hanya mengubah skala*
3. *Normalisasi wajib untuk algoritma berbasis jarak*
4. *Normalisasi tidak wajib untuk Decision Tree*

Dataset Akhir Siap Modeling

Setelah tahap ini, dataset:

- Sudah bersih
- Sudah numerik
- Sudah ternormalisasi
- Siap untuk:
 - K-Means
 - KNN
 - Clustering atau klasifikasi